



함수형 프로그래밍 맛보기

원정대장: 파라디

원정대원: 클라우디, 레스, 도넛, 유월, 루멘

목차

CHAP1. 함수형 프로그래밍이란?

CHAP2. 데이터 / 계산 / 액션

CHAP3. 계산 함수를 만들기 위해서는?

CHAP4. 실습

CHAP5. 원정대 활동 돌아보기

CHAP1.

함수형 프로그래밍이란?

CHAP1. 함수형 프로그래밍이란?

함수형 프로그래밍에 대하여

함수형 프로그래밍 이 뭘까?

같은 것을 넣으면 항상 같은 결과가 나오게 만들고,

실행했을 때 다른 곳에 영향을 주지 않도록 코드를 작성하는 방법

그래서 코드가 더 단순해지고, 문제가 생겨도 어디를 봐야 하는지 찾기 쉬워진다

CHAP1. 함수형 프로그래밍이란?

함수형 프로그래밍에 대하여

정확한 정의?

순수 함수

(수학 함수)처럼 입력값에 따라서만 결과가 결정되며,

부수 효과

(side effect)를 통제하는 프로그래밍 패러다임

CHAP1. 함수형 프로그래밍이란?

부수 효과? 순수 함수?

순수 함수

동일한 입력에 대해 항상 동일한 출력값을 반환 해주는 함수.

부수 효과(Side Effect)

프로그래밍에서 함수가 결과값을 반환하는 것 외에
함수 외부의 상태를 변경하거나 외부와 상호작용하는 모든 행위.

CHAP1. 함수형 프로그래밍이란?

함수형 프로그래밍의 목표

부수 효과를 통제하고

순수 함수를 지향한다.

CHAP2.

데이터 / 계산 / 액션

CHAP2. 데이터 / 계산 / 액션

우테코 판교 캠퍼스의 맛집은?



CHAP2. 데이터 / 계산 / 액션

액션일까? 계산일까?

“여러 메뉴 중 먹고 싶은 메뉴 **고르기**”

CHAP2. 데이터 / 계산 / 액션

데이터란

- 값 (숫자, 문자열, 객체, 배열 등)
- 이벤트에 대한 사실을 기록한 것.
- 그 자체로 실행되지 않음.

ex) 가진 돈, 메뉴



```
1  const rooms = [  
2    { id: "ROOM-A", name: "루비룸", capacity: 4, pricePerHour: 1000 },  
3    { id: "ROOM-B", name: "자바룸", capacity: 4, pricePerHour: 2000 },  
4  ];  
5
```

CHAP2. 데이터 / 계산 / 액션

계산이란

- 같은 입력을 넣으면 항상 같은 출력이 나오는 함수 (순수 함수)
- 호출 시점이 언제든, 몇 번을 호출하든 같은 결과를 반환

ex) 가진 돈으로 먹을 수 있는 점심 메뉴 구하기



```
1 function getPossibleMenus(amount, menus) {  
2   return menus.filter((menu) => (menu.price <= amount));  
3 }  
4  
5 const eatableMenus1 = getPossibleMenus(8000, menus); // [ ...  
6 const eatableMenus2 = getPossibleMenus(8000, menus); // [ ...
```

CHAP2. 데이터 / 계산 / 액션

액션이란

- 외부 세계에 영향을 주는 함수 (부수 효과)
- 호출 시점과 호출 횟수에 따라 결과값이 달라짐

ex) 메뉴 주문하기, 엘리베이터 타기, 음식 먹기



```
1 let orderCount = 0;
2
3 function order(menuName) {
4   orderCount += 1;
5   console.log(`${menuName} 주문완료! (${orderCount}번 째`);
6 }
7
8 order("볶음밥"); // "볶음밥 주문완료! (1번째)
9 order("볶음밥"); // "볶음밥 주문완료! (2번째)
```

CHAP2. 데이터 / 계산 / 액션

데이터 / 계산 / 액션

데이터

그 자체로 의미를 가지는 값이나 정보

계산

같은 입력이면 항상 같은 결과를 반환하는 함수

액션

실행할 때 마다 결과가 달라질 수 있거나, 외부 상태를 바꾸는 동작

CHAP2. 데이터 / 계산 / 액션

액션과 계산

액션 == 부수 효과

계산 == 순수 함수

CHAP2. 데이터 / 계산 / 액션

액션과 계산

액션에서 계산을 분리

액션을 계산으로 전환

CHAP2. 데이터 / 계산 / 액션

왜 액션을 계산으로?

1. 디버깅 및 테스트가 용이하다.
2. 재사용이 용이하다

함수형 프로그래밍

순수 함수(수학 함수)처럼 입력에만 따라 결과가 결정되며,

부수 효과를 통제하는 프로그래밍 패러다임

CHAP3.

계산 함수를 만들기 위해서는?

CHAP3. 계산 함수를 만들기 위해서는?

암묵적 출력 제거

암묵적 출력 제거: 함수가 외부 세계에 영향을 주지 않고, 변경한 내용을 **결과로 바로 반환하도록** 하는 것

```
1 let menuList = [
2   { name: "삼선짬뽕", price: 12000 },
3   { name: "볶음밥", price: 6000 },
4   { name: "유니짜장", price: 9000 },
5   {
6     name: "냉우동 (계절메뉴)",
7     price: 7000,
8   },
9 ];
10
11 function filterMenuList(money) {
12   const list = menuList.filter((menu) => (menu.price <= money));
13   console.log(`구매 가능한 메뉴리스트는 다음과 같습니다. ${list}`);
14 }
```

```
1 let menuList = [
2   { name: "삼선짬뽕", price: 12000 },
3   { name: "볶음밥", price: 6000 },
4   { name: "유니짜장", price: 9000 },
5   {
6     name: "냉우동 (계절메뉴)",
7     price: 7000,
8   },
9 ];
10
11 function filterMenuList(money) {
12   const list = menuList.filter((menu) => (menu.price <= money));
13   return list;
14 }
```

CHAP3. 계산 함수를 만들기 위해서는?

암묵적 입력 제거

암묵적 입력 제거: 함수가 외부 세계의 값을 바로 가져다 쓰지 않고, 필요한 값을 직접 받도록 바꾸는 것

```
1 let menuList = [
2   { name: "삼선짬뽕", price: 12000 },
3   { name: "볶음밥", price: 6000 },
4   { name: "유니짜장", price: 9000 },
5   {
6     name: "냉우동 (계절메뉴)",
7     price: 7000,
8   },
9 ];
10
11 function filterMenuList(money) {
12   const list = menuList.filter((menu) => (menu.price <= money));
13   return list;
14 }
```

```
1 let menuList = [
2   { name: "삼선짬뽕", price: 12000 },
3   { name: "볶음밥", price: 6000 },
4   { name: "유니짜장", price: 9000 },
5   {
6     name: "냉우동 (계절메뉴)",
7     price: 7000,
8   },
9 ];
10
11 function filterMenuList(menus, money) {
12   const list = menus.filter((menu) => (menu.price <= money));
13   return list;
14 }
```



CHAP3. 계산 함수를 만들기 위해서는?

부수 효과 제거



```
1  let menuList = [  
2    { name: "삼선짬뽕", price: 12000 },  
3    { name: "볶음밥", price: 6000 },  
4    { name: "유니짜장", price: 9000 },  
5    {  
6      name: "냉우동 (계절메뉴)",  
7      price: 7000,  
8    },  
9  ];  
10  
11 function filterMenuList(menus, money) {  
12   const list = menus.filter((menu) => (menu.price <= money));  
13   return list;  
14 }  
15  
16 const filteredList = filterMenuList(7000);  
17 console.log(`구매 가능한 메뉴리스트는 다음과 같습니다. ${filteredList}`);
```

CHAP3. 계산 함수를 만들기 위해서는?

실습

직접 코드로 확인해봅시다.
슬랙 링크 참고!

<https://functional-expedition.netlify.app>



CHAP4.

실습

CHAP4. 실습

실습에서 배운 것

1. 액션을 계산으로 쪼개고 분리하기

2. 계층형 설계

CHAP4. 실습

계층형 설계를 왜 해야 할까요?

계층형 설계를 왜 해야 할까요?

CHAP4. 실습

계층형 설계를 왜 해야 할까요?

1. 프로그램이 커져도 유지보수하기 쉬워진다
2. 같은 추상화 수준끼리 묶어서 읽기 쉬워진다
3. 역할과 책임이 분명해진다
4. 테스트하기 쉬워진다
5. 재사용하기 쉬워진다
6. 협업할 때 충돌이 줄어든다

CHAP4. 실습

(심화) 가변 데이터 / 불변 데이터

외부 상태 변경을 방지하기 위한 가장 확실한 방법! → 데이터 자체를 변하지 않게 만드는 것

- 가변 데이터: 생성된 이후에도 값 수정 가능
- 불변 데이터: 한 번 생성되면 그 상태를 바꿀 수 없는 데이터

그래서 원본 데이터를 직접 변경하는 대신 변경된 값을 가진 '새로운 복사본'을 생성하여 반환하는 방식 사용

- map, filter 등의 고차 함수 활용
- 카피 온 라이트

CHAP4. 실습

생각해보면 좋을 점

- 액션이 과연 나쁜걸까?
 - 모든 것을 불변으로 처리하고 부수 효과를 완전히 없애는 것은 현실적으로 불가능하며 비효율적이다.
- 실제로는 어떤 식으로 사용해보면 좋을까?

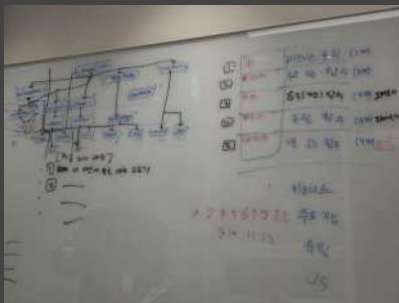
CHAP5.

원정대 활동 돌아보기

CHAP5. 원정대 활동 돌아보기

원정대 소개

- **목표:** 액션/계산/데이터를 기준에 따라 구분하고 계층형 설계를 할 수 있다.
- **원정대원:** 파라디, 레스, 루멘, 유월, 클라우디, 도넛(6명)
- **기간:** 3.9 ~ 3.20 (약 2주)
- **교재:** 쓱쓱 들어오는 함수형 코딩



CHAP5. 원정대 활동 돌아보기

타임라인

2026년 3월

< 오늘 >

월	화	수	목	금	토	일
9	10	11	12	13	14	15
🔥 [1차 원정대 활동]	🔥 [2차 원정대 활동]	👥 페어 프로그래밍			🔥 페어 프로그래밍 리뷰	
16	17	18	19	20	21	22
🔥 페어 프로그래밍 리뷰	🔥 원정대 활동 내용 정리 9:00	👥 페어 문제 설계	📚 테크 살롱 자료, 내용 마감	🔥 테크 살롱		
🌟 테크 살롱 플로우 구상 1	🔥 [3.5차 원정대 활동] 10:00	📚 캔버스 기획	📚 백엔드 피드백			
🔥 [3차 원정대 활동] 14:00	🔥 [4차 원정대 활동] 16:00	👥 홍보전략팀	🔥 [최종 원정대 활동] 1			
		🔥 [5차 원정대 활동]				

CHAP5. 원정대 활동 돌아보기

타임라인

원정대활동 기록

📅 캘린더 📋 리스트

Aa 이름

≡ 구분

📅 날짜

👤 [1차 원정대 활동]	원정대 활동	2026년 3월 9일
👤 [2차 원정대 활동]	원정대 활동	2026년 3월 10일
👤 [3차 원정대 활동]	원정대 활동	2026년 3월 16일 14:00
👤 [3.5차 원정대 활동]	원정대 활동	2026년 3월 17일 10:00
👤 [4차 원정대 활동]	원정대 활동	2026년 3월 17일 16:00
👤 [5차 원정대 활동]	원정대 활동	2026년 3월 18일
👤 [최종 원정대 활동] 1	원정대 활동	2026년 3월 19일
🎬 패어 프로그래밍	원정대 일정	2026년 3월 11일 → 2026년 3월 13일
🎬 패어 프로그래밍 리뷰	원정대 일정	2026년 3월 14일 → 2026년 3월 16일
📄 테크 살롱 자료, 내용 마감	원정대 일정	2026년 3월 19일
📄 테크 살롱	원정대 일정	2026년 3월 20일
🌟 테크 살롱 플로우 구상 1	개인 활동	2026년 3월 16일
📄 원정대 활동 내용 정리	개인 활동	2026년 3월 17일 9:00
📄 홍보전략팀	개인 활동	2026년 3월 18일
📄 캔버스 기획	개인 활동	2026년 3월 18일
📄 패어 문제 설계	개인 활동	2026년 3월 18일
📄 백엔드 피드백	개인 활동	2026년 3월 19일

CHAP5. 원정대 활동 돌아보기

활동 방식1. 개념 공부: 책 읽으며 끄적이고 지식 전이하기

#책을 읽으면서 끄적이기

▶ CHAP3

▼ CHAP4

레벨1에서 중요시하는 것과 굉장히 유사

- 관심사 분리, ui 로직 분리 등의 내용이 현재 우리가 미션에서 초점을 맞추는 것과 굉장히 유사함

암묵적 입력, 암묵적 출력 (p70~p78)

- 암묵적 입력과 암묵적 출력까지 구분을 해야할까? 당연히 전역변수 대신 매개변수로 넘겨야 하고 당연히 함수에 리턴을 통해서 기능을 해야한다고 생각했는데 개념적으로 명시를 할지(때때로 머리 써서 코드를 지킬까만해서)지 구분을 해야하나 생각이 들었다.
- 어떤 함수면 암묵적 출력과 입력이 있어야(최소화되어야)한다
 - 언자가 아닌 모든 입력(변수 등)은 암묵적 입력이고, 리턴값이 아닌 모든 출력(console)등은 암묵적 출력이다

▶ CHAP5

#활동 때 뭘 하면 좋을까

- 본인이 과거에 했던 코드(프로그래밍, 미션 혹은 그 이전 등) 중 하나를 골라서 데이터, 계산, 액션 나눠서 리팩토링 해보고 코멘트에 공유하기 (무 복잡하지 않게, 코멘트가 이해할 수 있는 수준으로, 직접 해보는 것에 의미)
- 역선으로부터 이루어진 일인가 해 있는 하나의 예제 (시도 생성)에서 계산을 빼내 리팩토링해보고 각자 결과물 공유 (실습 연습, 다른 사람과 교방식 공유 후 인사이트 넓히기 목적) (70p)
- p76(slice 메소드([...arr])와 복사 후 return)에 대해 토론해보기
- 특정 액션(or 계산)을 임시 형태(필자 단위)로 최대한 쪼개보기(99p) → 누군가가 더 쪼갤 수 없을 때까지
 - 그랬을 때의 장단점과 어느 정도로 쪼개야 적당한지 예기해보기

#공유회 때 뭘 하면 좋을까

- 역선, 계산, 데이터는 물리에서 원자와 같은 것이다.
 - 어떤 분자를 원자 단위로 쪼갬을 때 해당 원자들의 조합으로 더 많은 분자를 만들 수 있듯이 어떤 기능 집합을 역선, 계산, 데이터로 분리할 때 분리된 것들의 조합으로 더 다양한 것을 쉽고 확실히 있게 만들 수 있다.
- 역선은 전명성이 있다 (56p)

한줄 요약: 아... 대체 어디까지 만족할만큼 쪼개야하는거, 계산으로 어디까지 바꾸는게 맞는가...

동작 구분: 다시 사람(시도)해도 같은 결과가 나오는가 vs 다른 결과가 나오는가

결국 액션을 최소화 하고 계산으로 최대한 풀어내는 것.
왜? 만들수있는 함수가 많아질 수록 신경써야 하는 범위가 줄어들기 때문.

→ 쉽게 이해하면 외부 세계의 변화를 인수로 받으면 계산, 함수 내부에서 직접 외부를 건드리면 액션

시간에 따라 바뀌는 값을 모델링 할 때에 동작 방법을 이해하는것은 중요하지만 어렵다. → 실행 시점이나 함수에 의존하는 코드를 없애면, 코드를 더 쉽게 이해할 수 있고 심각한 버그를 막을 수 있다.

41p "무슨데 등급 매기기" 아무리봐도 계산같은데 (아직 해설 안읽어봄)

해설 읽기전에 생각해보자만... 아마 { coupon: "10%", rank: "good" } 이런식으로, 등급을 매기는 행위가 아니라 이미 등급이 적혀있는 쿠폰 데이터라서 데이터로 분류한게 아닐까

맞는듯

이때일 목적을 계획하는 계산을 더 작은 계산으로 나눠보는 part

- 쿠폰 목록에서 'good' 쿠폰 목록과 'best' 쿠폰 목록을 계산 할 수 있다.
- 어떤 구독자가 'good' 쿠폰을 받는지 'best' 쿠폰을 받는지 결정하는 것을 계산으로 만들 수 있다 (rec_count > 10)

▶ 내 코드로 생각해보자

DOM 업데이트와 비즈니스 규칙은 분리되어야 한다.

전역변수를 읽는 것은 암묵적 입력이고 바꾸는 것은 암묵적 출력이다. 따라서 전역변수가 없어야하고 → 암묵적 입력은 인자로 바꾸고 암묵적 출력은 리턴값으로 바꾸되자. (비슷: 전역변수에 의존하지 않아야 한다)

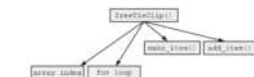
서브루틴 (extract subroutine) 추출하기

7 '직접 구현'한다는 것이 의미?

함수의 의도(비즈니스 로직 등)를 표현할 때, 계층을 건너뛰어 너무 낮은 수준의 기능 (for문, 배열 인덱스 접근 등)을 여러 계층의 함수들만 사용하여 해당 로직을 '직접적으로' 구성하려는 뜻

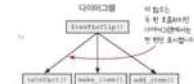
→ 함수 내부의 모든 동작이 동일한 추상화 수준을 가지도록!

- 흐름 그래프를 만들어 함수 간의 계층을 표현하고 함수 흐름을 시각화하기
 - before



동작논리

- after



모두 같은 일이라 함수로 만들거

- 흐름 그래프에서 같은 색을 가리킨다는 것은 같은 계층에 있어도 좋다는 의미
- 계층형 설계는 코드를 추상화 계층으로 구성
 - 각 계층을 볼 때 다른 계층의 구체적인 내용은 몰라도 됨

같은 계층에 있는 함수는 같은 목적을 가져야 한다.

- 계층형 설계에서 모든 계층은 바로 아래 계층에 의존해야 한다.

7 높은 수준 / 낮은 수준의 함수

함수가 높은 수준일수록 비즈니스 언어에 가깝고, 낮은 수준의 함수일수록 컴퓨터와 연관이 깊어진다

패라디

레스

유월

CHAP5. 원정대 활동 돌아보기

활동 방식1. 개념 공부: 책 읽으며 끄적이고 지식 전이하기

액션, 데이터, 계산 분리시 장점

→ 분산 시스템에 잘 어울린다

- 액션 코드를 돌리면, 코드를 더 쉽게 이해할 수 있다.
- 데이터나 계산은 실행 시점이나 횟수에 의존하지 않아 분산 시스템에 생기는 여러 문제를 해결할 수 있다.

계층형 설계

- 아키텍처 패턴이 계층을 만들기 때문에 계층형 설계라고 부른다
- 일반적으로 비즈니스 규칙, 도메인 규칙, 기술 스택으로 나눈다 (왼쪽으로 갈수록 자주 바뀌지 않는 코드)

타임라인 다이어그램

- 분산 시스템을 타임라인 다이어그램으로 표시하면, 문제를 파악하는데 도움이 되었지만 실행 순서가 섞여 보이는 문제가 발생.
 - 커링을 사용하여 여러 타임라인이 진행될 때 순서를 맞춰서 이를 해결

액션, 계산, 데이터를 구분에 대해 더 알아보자

적용 사례

- 문제에 대해 생각할 때
 - 문제 설계 시 구분하면 코드를 볼 때 주의해야 할 부분에 대해 미리 알고 싶을 있다.
- 코딩할 때
 - 최대한 액션에서 계산(순수 함수)을 빼려고 하자
 - 액션에서 계산을 뺄 수 있는지 고민, 반대도 같이 고민
- 코드를 읽을 때
 - 액션은 실행 시점에 의존하기 때문에 더 조심해야 함
 - 숨어있는 액션을 찾지?
 - 냉장고 확인을 냉창고 확인, 냉동고 확인으로 나눠 액션을 나눌 수 있음

계산(순수 함수)을 발견하기 어려운 이유

- 일상 생활 속 감정해야 할 일들은 자연스럽게 머릿속에 떠오르기 때문
- 그러나 함수형 프로그래밍을 잘 하기 위해서는 의식적으로 연습이 필요

추상화의 벽

- 세부 구현을 강준 함수로 이루어진 계층, 하나의 인터페이스의 역할을 한다.
- 이 함수들을 사용할 때는 구현을 전혀 몰라도 함수를 쓸 수 있다.
- 추상화의 벽을 사용하면 각 팀의 역할만 수행해 협업이 용다.

(어디든 추상화의 벽 레이어가 될 수 있는 것 아닌가? 정확히 어느 계층을 이어가 하는가? => 상황이다, 기준이다 다를 수 있다. 계층을 어떻게 나눈다나 따라서 다름, 용인 인터페이스 계층을 두는 것으로 이해.)

- 벽을 넘어서 사용하는 경우는 없어야 한다. (추상화의 벽과 그 위, 추상화의 벽과 그 아래에만 호출요가 있어야 한다. 가로지를 수 있다.)
 - 한약 가로지르는 확실도가 생기기, 필요한 함수를 추상화의 벽에 하나 더 만들면 해결된다.

언제 써야할까?

- 구현에 대한 확신이 없는 경우, 구현을 간접적으로 사용할 수 있기 때문에 나중에 구현을 바꾸기 쉽다
- 바뀔 것을 알고 있지만 아직 준비가 되지 않은 경우에 좋다.
 - 물론 너무 미래를 생각해서 작성하지는 말아라.
- 코드를 읽고 쓰기 쉽게 만들기 위해
 - 상위에서 세부적인 것을 신경쓰지 않아도 된다.
 - 오프 바이 원, 반박문 초기화 등등
- 팀 간에 조율해야 할 것을 줄이기 위해
 - 각 팀에 관한 구체적인 내용을 서로 신경 쓰지 않아도 될 수 있다.
- 주어진 문제를 집중하기 위해
 - 해결하려는 문제의 구체적인 부분을 무시할 수 있다.

코드를 쓸 수는 중요하지 않다.
중요한 것은 코드가 적절한 추상화 수준과 일반화가 되어 있는지이다.
한줄짜리 코드는 여러 구체화 수준이 섞일 일이 없기 때문에 좋은 코드이다.

작은 인터페이스

인터페이스를 최소화하면 하위 계층에 불필요한 기능이 불려들이 커지는 것을 막을 수 있다.

새로운 기능을 만들 때 하위 계층의 기능을 추가하거나 고치는 것보다 상위 계층에 만드는 것을 말한다.

상위 계층에 어떤 함수를 만들 때 어떤 가능한 현재 계층에 있는 함수로 구현하는 것이 작은 인터페이스를 실현하는 방법이다.

→ 액션, 계산, 데이터 구분

- 계산 - 입력값을 받아 출력값을 만드는 연산
 - 수학의 순수 함수와 동일
 - 동일한 입력에는 항상 동일한 출력을 반환
 - 외부에 영향을 주지 않음(부수 효과 없음)
 - 테스트하기 매우 쉬움
- 액션 - 호출하는 시점과 횟수에 의존
 - 이메일 보내기, 데이터베이스 읽기/쓰기, DOM 업데이트 등
 - 부수 효과를 동반, 실행 순서가 중요
 - 테스트하기 어려움

액션은 계산으로, 계산을 데이터로

CH 4 액션에서 계산 빼내기

- 인자가 아닌 모든 입력은 암묵적 입력
- 리턴값이 아닌 모든 출력은 암묵적 출력

- 암묵적 입력과 출력 찾기
- 계산 추출하기

CH 5 더 좋은 액션 만들기

- 액션 최소화하기
- 응용도 높이기, 결합도 낮추기

- 분리된 것은 언젠게 쉽게 조합할 수 있다
- 오히려 잘 분리하는 방법을 찾기 더 어렵다
- 코드 라인 수만으로 좋은 코드인지 판단하기 어려움

루멘

도넛

클라우드

CHAP5. 원정대 활동 돌아보기

활동 방식2. 몸에 익히기: 페어 프로그래밍 + 코드 리뷰

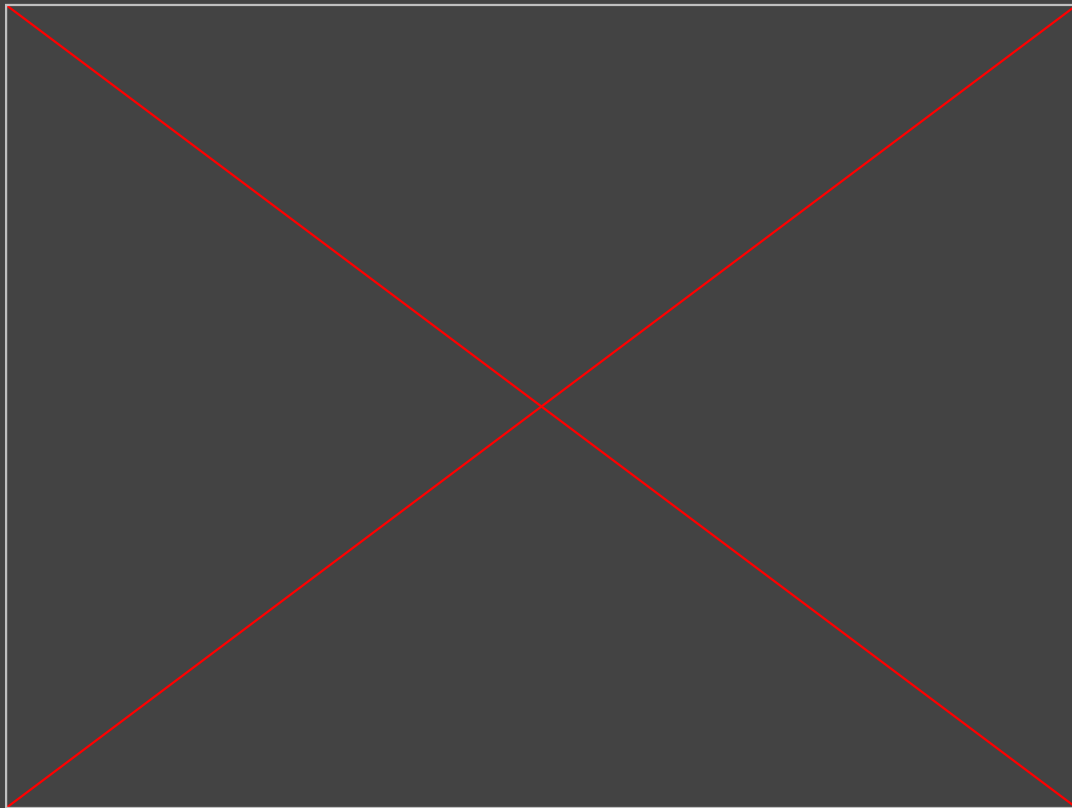
2026-FP-DEEP-DIVE

스터디 목적

'쑹쑹 들어오는 함수형 코딩' 도서를 기반으로 함수형 프로그래밍의 특징을 학습하며 액션-계산-데이터의 분리, 불변성 유지, 부수 효과 통제 및 선언 목적으로 합니다.

스터디 구성원

팀	팀원	팀원
Pair 1	 파라디	 래스
Pair 2	 루엣	 클라우드
Pair 3	 요윳	 도넛



CHAP5. 원정대 활동 돌아보기

활동 방식3. 논쟁 + 송곳 만들기

1. 페어 프로그래밍에서 나온 애매한 지점들을 모아서 토론
2. 같은 개념을 공부했지만 서로서로 의견이 다른 지점이 발생했음.

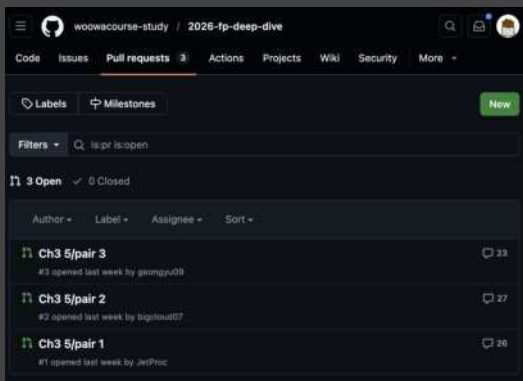
throw를 사용하면 액션인 것일까? throw는 암묵적 출력인가?

변하지 않는 상수임에도 암묵적 입력으로 받아오면 액션인가?

CHAP5. 원정대 활동 돌아보기

그래서 얻은 게 뭘데?

정량적 결과



정성적 결과

1. 계산/액션/데이터를 분리 할 수 있다
2. 순수 함수, 암묵적 입출력 등에 대한 개념을 정확히 알 수 있다
3. 계층형 설계 능력

CHAP5. 원정대 활동 돌아보기

개인 회고

파라디

프로그램 설계 시점에서 계산/액션/데이터를 인식적으로 구분할 수 있게 되었고, 페어 프로그래밍/토의/원정대 발표 준비까지 단순히 지식 말고도 여러가지를 얻을 수 있었던 시간이었습니다.

유월

무의식적으로 하고는 있었지만, 모호하게 알고 있었던 함수 프로그래밍 패러다임에 대해 제대로 알 수 있어 좋았습니다. 무엇보다 열정적인 크루들과 함께해서 재밌었어요 🍌

클라우디

파편적으로 흩어져 있던 함수형 프로그래밍에 대한 개념을 단단하게 확립할 수 있는 뜻깊은 시간이었습니다. 무엇보다 페어 프로그래밍과 상호 간의 코드 리뷰를 거치며 크루들과 활발하게 의견을 교환할 수 있어 더욱 즐거웠습니다.

루멘

들어는 봤지만 자세하게는 몰랐던 함수형 프로그래밍에 대해 조금 더 알 수 있던 시간이었습니다. 원정대를 진행하고 테크 살롱을 준비하면서 즐겁고 깊게 나만의 송곳을 만들 수 있어서 좋았고 적극적이고 아이디어가 넘치는 팀원들을 만나 행복한 시간이었습니다.

도넛

언젠가는 함수형을 제대로 공부하고싶다는 생각을 가지고 참여했고, 함수형의 핵심을 배울 수 있어서 좋았습니다! 함수형에 대한 개념을 공부하기도 했지만, 페어프로그래밍이나 발표 준비 등 소프트스킬까지 배울 수 있는 값진 시간이었습니다!

레스

다른것보다 이 함수는 액션인지 계산인지를 인식하기 시작하니 코드를 짜는 방식이 확실히 달라졌습니다. 월 분리해야하고 월 인자로 받아야하고, 월 리턴해야하는지가 명확해졌다고 해야할까요. 모두가 열정적인 스터디였기에 너무 즐거웠습니다.



많은 피드백 부탁드립니다!!

